

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Name: Atiksh Borundia

Roll No.: 24011101009

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are:

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment, the following outcomes are demonstrated:

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks automatically learn hierarchical image features and have become important models for computer vision. The evolution of CNN architectures has focused on improving classification accuracy while addressing training difficulty and computational cost.

The experiment compares shallow CNNs trained from scratch with ImageNet-pretrained architectures used through transfer learning. The primary transfer-learning model is VGG16, with additional experiments using ResNet50, InceptionV3 as an Inception-family substitute, and MobileNetV2.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was implemented from scratch for this experiment. It uses convolution, average pooling and fully connected layers. It contains 83,126 trainable parameters.

After 10 epochs, LeNet-5 achieved a training accuracy of 62.85% and a test accuracy of 50.24%. Its training time was 69.5 seconds.

6. AlexNet

The original AlexNet requires larger input images, so an adapted CIFAR-10 version was implemented using smaller convolutional settings..

After 10 epochs, the adapted AlexNet achieved 77.68% training accuracy and 69.74% test accuracy. It contained 7,506,762 parameters and required 202.3 seconds of training.

7. VGG16

VGG16 was used as the primary transfer-learning model. The CIFAR-10 images were resized to 96×96 before being passed to the pretrained backbone. ImageNet weights were loaded, the original classifier layer was replaced, and a Global Average Pooling layer, Dense layer and Softmax output layer were added.

The feature-extraction phase kept the convolutional base frozen. The model was then fine-tuned by making the final VGG16 convolutional block trainable.

8. Comparison of Architectures

The theoretical comparison specified by the laboratory manual is:

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

The experimental parameter counts differ from the original values due to changes in the implementations.

9. GoogleNet (Inception Network)

InceptionV3 transfer learning achieved 85.81% training accuracy and 78.87% test accuracy. Its reported parameter count was 22,329,898 and training time was 474.7 seconds.

10. ResNet

ResNet addresses optimization difficulties in very deep networks through residual connections. Instead of directly learning $H(x)$, the residual formulation learns

$$F(x) = H(x) - x$$

and produces

$$\text{Output} = F(x) + x.$$

In this experiment, ResNet50 was used with transfer learning and subsequent fine-tuning. It achieved 52.32% training accuracy and 46.29% test accuracy, with precision 0.5519, recall 0.4629 and F1-score 0.4447.

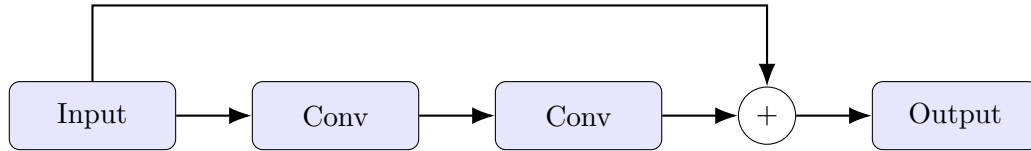


Figure 1: Residual Block in ResNet

11. Dilated Convolution

Dilated convolution, also called atrous convolution, increases the receptive field by inserting gaps between kernel elements without increasing the number of kernel parameters. It is useful in applications such as semantic segmentation and object localization.

For standard convolution, $D = 1$, whereas dilated convolution uses $D > 1$.

12. Transpose Convolution

Transpose convolution performs learnable upsampling and increases the spatial dimensions of feature maps. It is commonly used in image generation, autoencoders, GANs, super-resolution and semantic segmentation.

13. Transfer Learning

Transfer learning reuses visual features learned by a model trained on a large dataset such as ImageNet. The workflow used in this experiment is:

1. Load a pretrained model with ImageNet weights.
2. Replace the original classifier.
3. Freeze the convolutional base.
4. Add Global Average Pooling and Dense classification layers.
5. Train the new classifier.
6. Unfreeze the last convolutional block and fine-tune it.

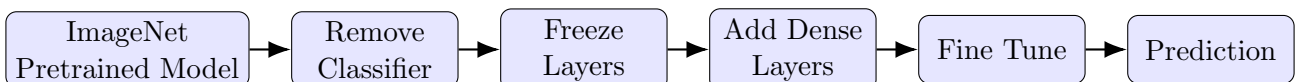


Figure 2: Transfer Learning Workflow

14. Dataset

The experiment used CIFAR-10. The notebook loaded 50,000 training images and 10,000 test images, then split the original training data into 45,000 training and 5,000 validation images.

- Training set: $45,000 \times 32 \times 32 \times 3$
 - Validation set: $5,000 \times 32 \times 32 \times 3$
 - Testing set: $10,000 \times 32 \times 32 \times 3$
 - Number of classes: 10
 - Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck
- Pixel values were normalized to the range $[0, 1]$.

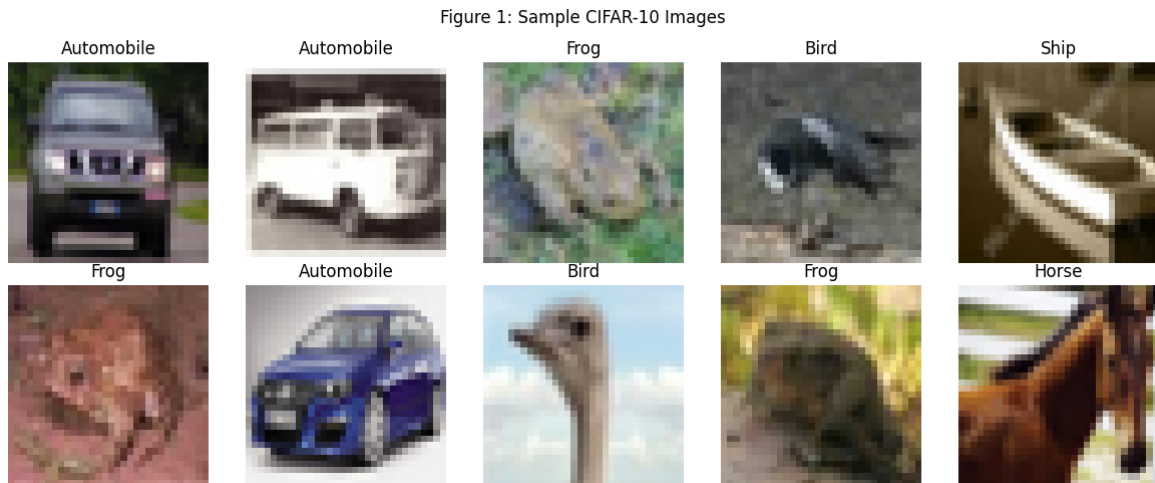


Figure 3: Sample CIFAR-10 Images

Inference: The samples confirm the use of low-resolution 32×32 RGB images across all ten CIFAR-10 classes. The low resolution also makes visually similar categories harder to distinguish.

15. Experimental Procedure

Task 1: Dataset Preparation

CIFAR-10 was loaded from the dataset files, normalized, split into training, validation and test sets, and ten sample images were displayed.

Task 2: Transfer Learning

VGG16 was selected as the primary pretrained CNN. Its ImageNet-trained convolutional base was frozen initially, and the original classifier was replaced with an experiment specific classification layer consisting of Global Average Pooling, a 256-unit Dense layer, Dropout and a 10-class Softmax output.

Task 3: Model Training

The primary VGG16 feature-extraction phase used Adam with learning rate 0.001, batch size 32, categorical cross-entropy loss and 10 epochs.

The feature-extraction phase achieved 70.15% test accuracy.

Task 4: Fine Tuning

The last convolutional block of VGG16 was unfrozen and trained for five additional epochs using a lower learning rate. Test accuracy increased from 70.15% before fine-tuning to 81.88% after fine-tuning, an improvement of 11.73 percentage points.

Task 5: Model Evaluation

The final VGG16 model was evaluated using accuracy, macro precision, macro recall, macro F1-score, confusion matrix and a classification report.

16. Hyperparameter Study

The laboratory manual specifies a study over learning rate $\{0.001, 0.0001\}$, batch size $\{16, 32, 64\}$, epochs $\{10, 20\}$, optimizer $\{\text{Adam}, \text{SGD}\}$, dense units $\{128, 256\}$ and freezing strategy $\{\text{All}, \text{Partial}\}$. This corresponds to 48 possible combinations.

LR	Batch	Epochs	Opt.	Dense	Frozen	Test Acc.	Time (s)
0.0001	64	20	SGD	128	All	48.86%	1227.5
0.0001	64	10	Adam	128	Partial	86.14%	753.5
0.001	64	10	SGD	128	Partial	85.01%	715.4
0.001	16	10	Adam	128	Partial	10.00%	1111.8
0.001	64	10	Adam	128	All	68.43%	615.9

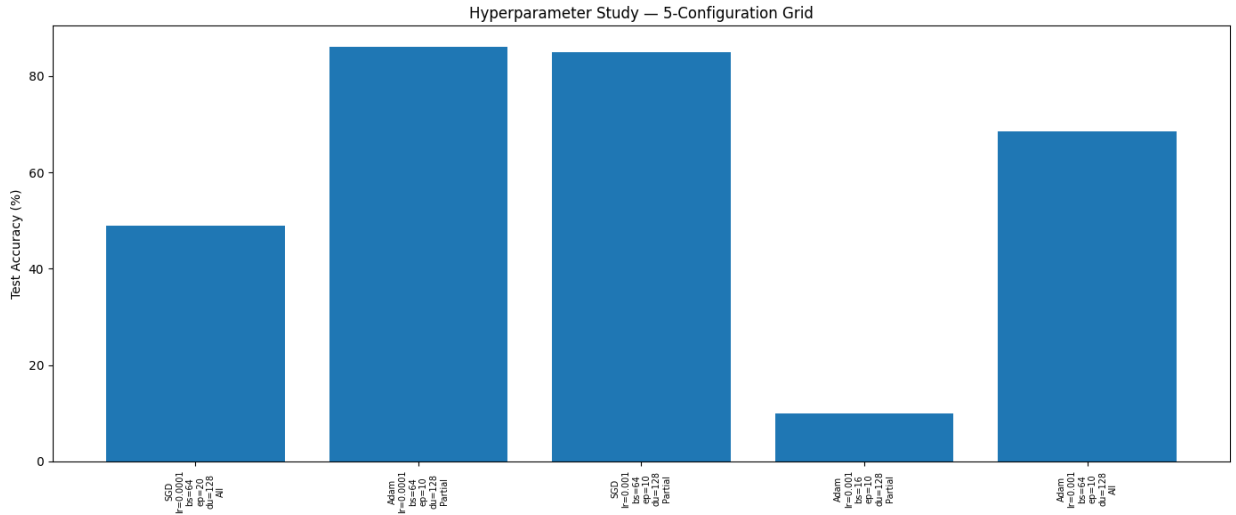


Figure 4: Executed Hyperparameter Study

Inference: Among the five executed configurations, the Adam configuration with learning rate 0.0001, batch size 64 and partial freezing produced the highest test accuracy of 86.14%. The results also show that optimizer, learning rate and freezing strategy and all the other hyperparameters can strongly affect transfer-learning performance. So, hyperparameter optimization is very important.

17. Mandatory Plots

17.1 Training Accuracy

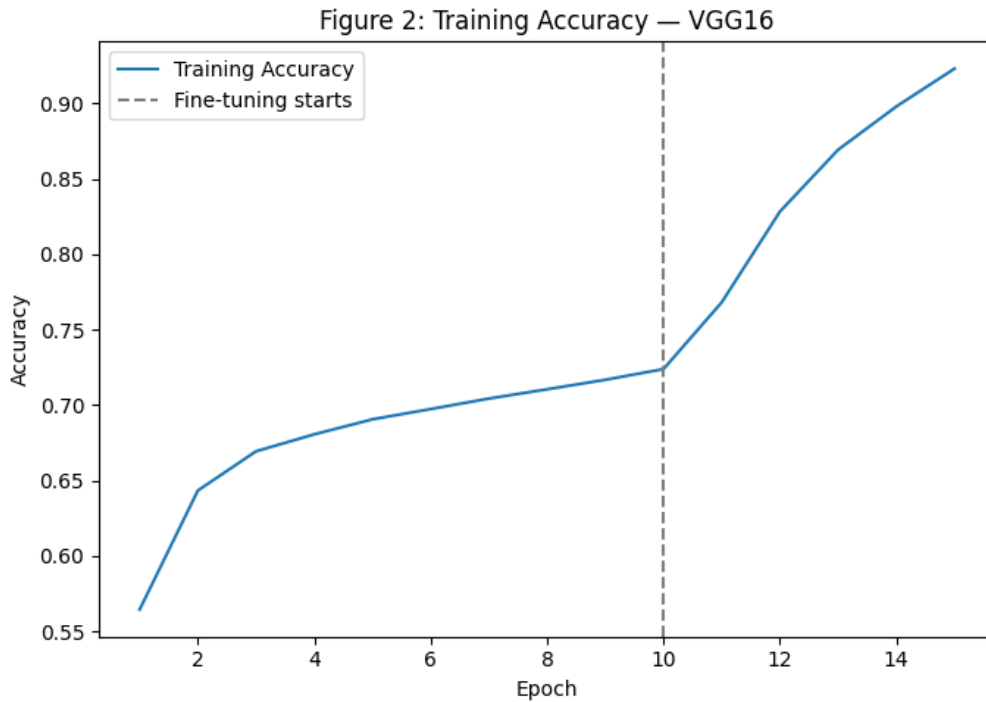


Figure 5: Training Accuracy of the Primary VGG16 Model

Inference: Training accuracy has increased throughout feature extraction and improved further after fine-tuning began, reaching 92.30%. This shows that unfreezing the final convolutional block allowed the model to adapt more closely to CIFAR-10 and forget the pretrained part of the model.

17.2 Validation Accuracy

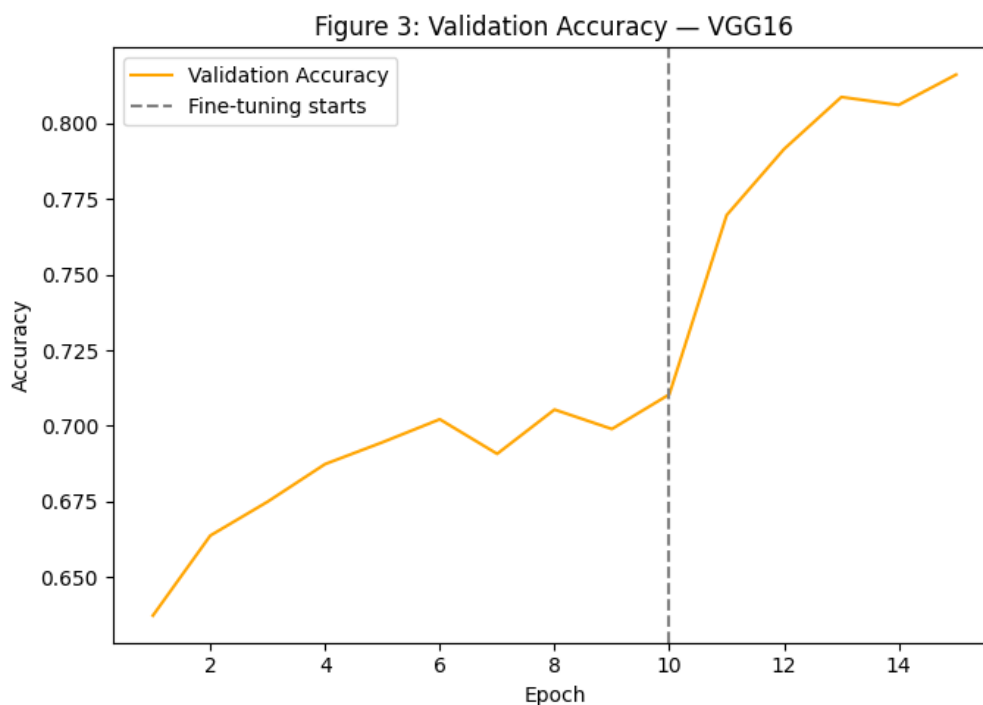


Figure 6: Validation Accuracy of the Primary VGG16 Model

Inference: Validation accuracy improved from the feature-extraction phase and reached 81.60% at the end of fine-tuning. The validation curve remains reasonably close to the training trend, although a generalization gap is visible as it is less than the training accuracy achieved earlier.

17.3 Training Loss

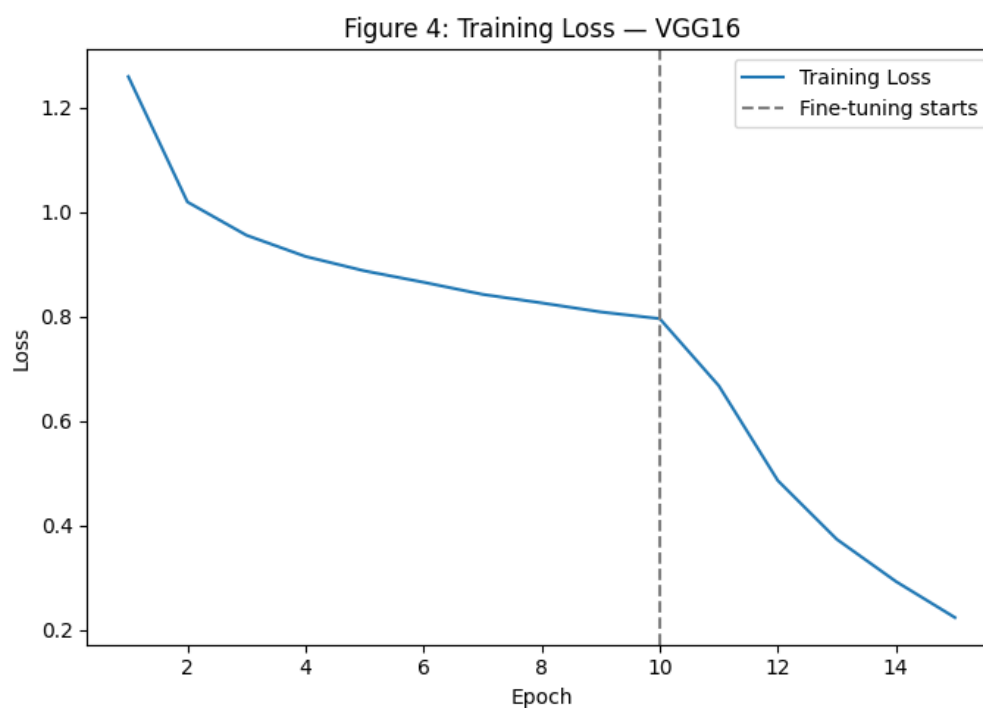


Figure 7: Training Loss of the Primary VGG16 Model

Inference: Training loss decreases very much after fine-tuning begins, indicating improved adaptation of the pretrained features to the target classification task.

17.4 Validation Loss

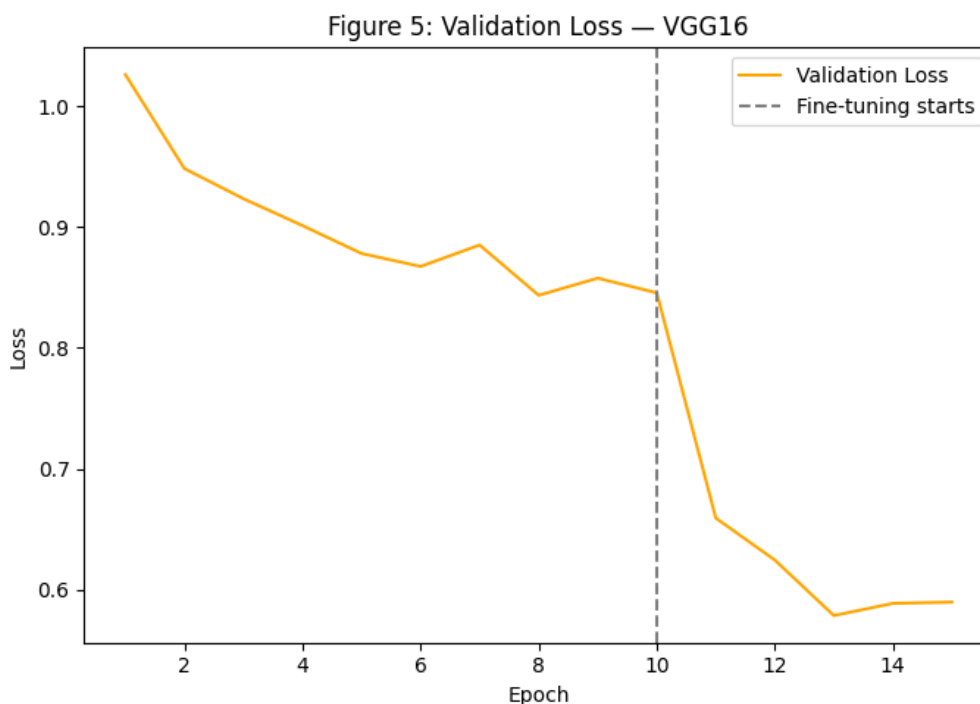


Figure 8: Validation Loss of the Primary VGG16 Model

Inference: Validation loss decreases overall, although small fluctuations occur during fine-tuning. These fluctuations indicate that additional training could eventually require regularization or early stopping as at the end the validation accuracy starts to plateau even though training loss decreases.

17.5 Confusion Matrix

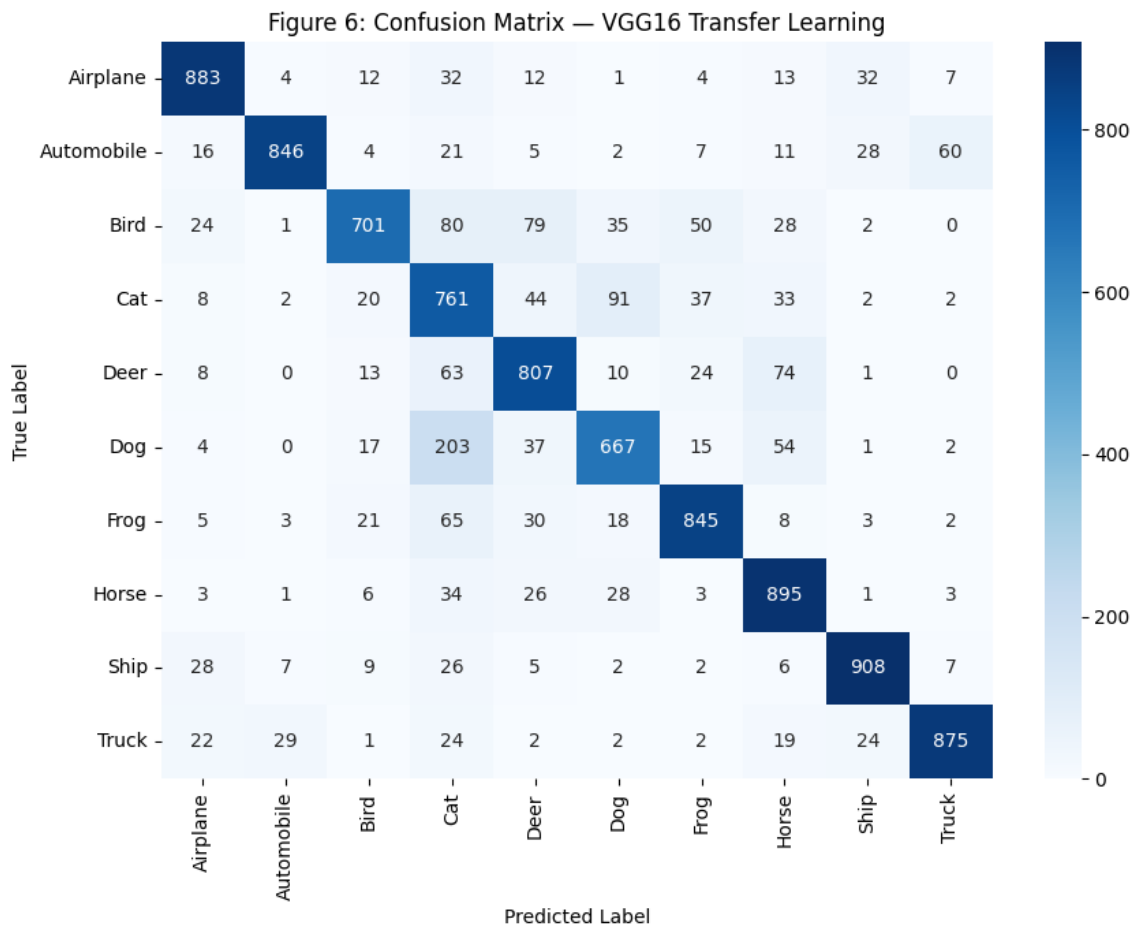


Figure 9: Confusion Matrix for Fine-Tuned VGG16

Inference: The strong diagonal indicates that most test images were classified correctly, but the model misclassified similar looking animals due to low resolution images provided in the dataset.

17.6 Misclassified Images

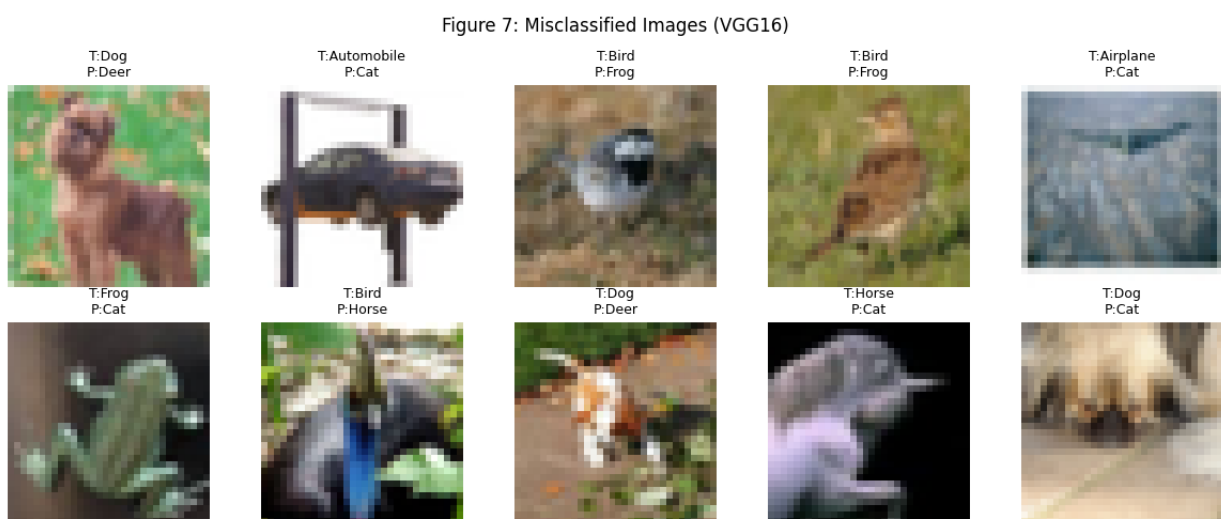


Figure 10: Examples of Misclassified CIFAR-10 Images

Inference: The notebook identified 1,812 images wrongly out of 10,000 test images. The above images are a few example of visually similar looking images being misclassified and also visually different looking images.

18. Source Code

The complete source code used for implementing the CIFAR-10 experiments, transfer learning, fine-tuning, hyperparameter experiments and model evaluation is available in the following Google Colab notebook:

<https://colab.research.google.com/drive/1ajFu-S3ifoKXrrVBVxWyu9U0u5i8jQxh?authuser=3#scrollTo=19133c0d>

Google Colab Notebook: *Experiment 4 – CIFAR-10 Transfer Learning*

19. Results

19.1 Performance Metrics – Primary VGG16 Model

Metric	Value
Training Accuracy	92.30%
Testing Accuracy	81.88%
Precision	0.8291
Recall	0.8188
F1-score	0.8207
Training Time	1207.1 s
Total Parameters	14,848,586

The classification report shows the highest class-level F1-score for Ship (0.91), followed by Automobile and Truck (0.89 each). Cat had the lowest class-level F1-score (0.66).

19.2 Comparison of CNN Architectures

Model	Train Acc.	Test Acc.	Precision	Recall	F1	Time (s)
LeNet-5	62.85%	50.24%	0.5123	0.5024	0.5018	69.5
AlexNet (adapted)	77.68%	69.74%	0.6999	0.6974	0.6960	202.3
VGG16 (fine-tuned)	92.30%	81.88%	0.8291	0.8188	0.8207	1207.1
ResNet50 (fine-tuned)	52.32%	46.29%	0.5519	0.4629	0.4447	694.8
InceptionV3	85.81%	78.87%	0.7939	0.7887	0.7894	474.7
MobileNetV2	86.58%	80.78%	0.8110	0.8078	0.8074	190.4

Model	Total Parameters
LeNet-5	83,126
AlexNet (adapted)	7,506,762
VGG16 (fine-tuned)	14,848,586
ResNet50 (fine-tuned)	24,114,826
InceptionV3	22,329,898
MobileNetV2	2,588,490

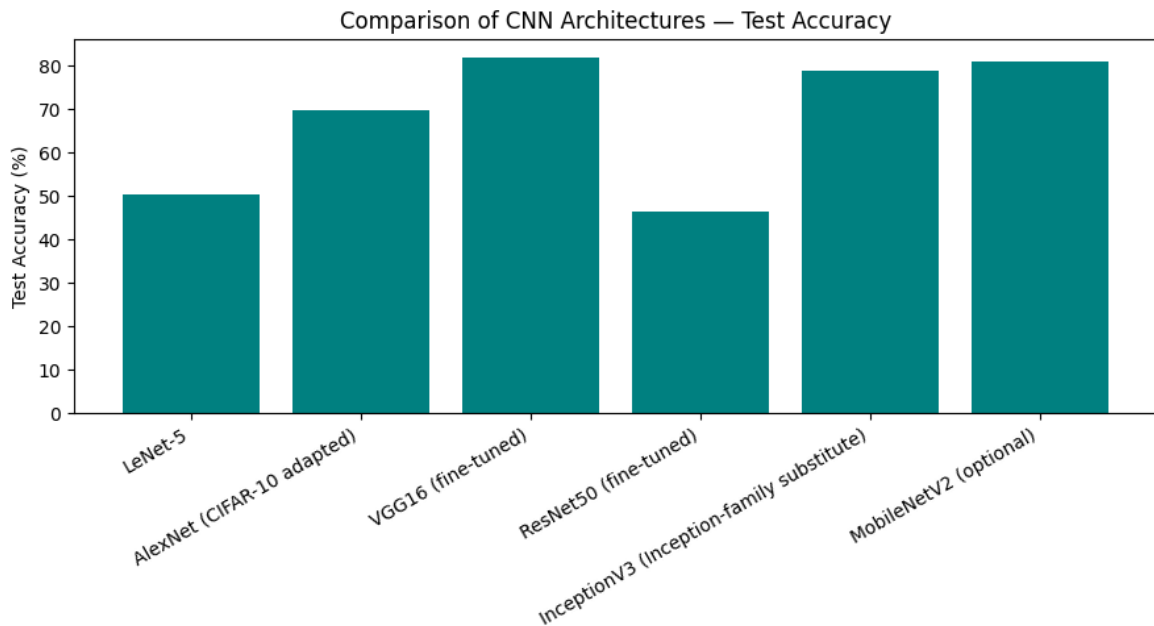


Figure 11: Test Accuracy Comparison of the Executed Architectures

Overall inference: Transfer-learning models generally outperformed the scratch-trained LeNet-5 and adapted AlexNet models. VGG16 achieved the highest test accuracy in the main architecture comparison, while MobileNetV2 provided a strong accuracy of 80.78% with fewer parameters, while Resnet showed very bad performance over the dataset.

20. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**
AlexNet won the 2012 ImageNet competition by a large margin in the early years of deep learning and popularized ReLU activations, Dropout and GPU-accelerated CNN training.
2. **Why does VGG16 use only 3×3 convolution filters?**
Stacking 3×3 filters provides a receptive field comparable to larger filters while using fewer parameters and introducing additional non-linearities. It is the perfect size with small size and adding non-linearities as well.
3. **Explain the advantages of the Inception module.**
It uses parallel convolutions and pooling. It reduces computational cost and dimensionality if lower kernel size used.
4. **What is the purpose of residual learning?**
They solve the vanishing gradient problem by learning a residual mapping $F(x) = H(x) - x$ through the neural networks.
5. **Differentiate LeNet and ResNet.**
LeNet-5 is a small-parameter CNN intended for simple image recognition, whereas ResNet is much deeper and uses residual connections to enable stable training of deep networks.
6. **What is Transfer Learning?**
Transfer learning reuses features of a pretrained model, such as an ImageNet model, for a new target task. The classifier layer is replaced and the pretrained features can be frozen or fine-tuned.
7. **Why is fine tuning required?**
Pretrained features are general-purpose. Fine-tuning the later layers adapts the connection layers to the target dataset and can improve classification performance.
8. **Explain the difference between dilated convolution and transpose convolution.**
Dilated convolution expands the receptive field by inserting gaps between kernel elements, whereas transpose convolution performs upsampling and increases spatial dimensions instead of decreasing them.

9. Why do pretrained models converge faster?

They have already the weights for the feature extraction part and can be generalized later for specific tasks.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has very few parameters and low computational cost, whereas ResNet50 has millions of parameters and substantially higher computation. The greater cost enables a much deeper feature representation.

21. Additional Exercises

The notebook additionally implemented:

1. Transfer learning using VGG16.
2. Transfer learning and fine-tuning using ResNet50.
3. An Inception-family experiment using InceptionV3.
4. Transfer learning using MobileNetV2.
5. Evaluation using Precision, Recall and F1-score.
6. Comparison of scratch-trained and pretrained architectures.

The executed results demonstrate that the choice of pretrained backbone matters substantially for CIFAR-10. In this run, VGG16 and MobileNetV2 performed particularly strongly, while the ResNet50 configuration used in the notebook performed substantially worse.

22. Conclusion

So to conclude, 5 types of CNN Models were tried on the CIFAR 10 dataset with a pretrained VGG as well. The VGG showed the highest accuracy and the ResNet showed the lowest accuracy. Transfer Learning enables the model to learn faster and converge earlier than starting from scratch with pre-trained weights for the features extraction part and then fine tuning the classification layer to increase the accuracy even further.

22. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>